

城小二校园外卖系统 - 代码清单

目录

- 一、前端代码清单（微信小程序）
- 二、后端API接口清单
- 三、管理端代码清单（Vue3）

一、前端代码清单（微信小程序）

1.1 购物车管理代码

文件位置: `store/cart.js`

```
import { defineStore } from 'pinia'

export const useCartStore = defineStore('cart', {
  state: () => ({
    cartList: []
  }),

  getters: {
    // 判断购物车是否为空
    isEmpty: (state) => state.cartList.length === 0,

    // 计算购物车总数量
    totalCount: (state) => {
      return state.cartList.reduce((total, item) => total + item.quantity, 0)
    },

    // 计算购物车总金额
    totalAmount: (state) => {
      return state.cartList.reduce((total, item) => {
        return total + item.price * item.quantity
      }, 0)
    }
  },

  actions: {
    // 添加商品到购物车
    addGoods(goods) {
      const index = this.cartList.findIndex(item => item.goods_id === goods.goods_id)
      if (index !== -1) {
        this.cartList[index].quantity++
      } else {
        this.cartList.push({
          goods_id: goods.goods_id,
```

```
        name: goods.name,
        price: goods.price,
        image: goods.image,
        quantity: 1
      })
    }
    this.saveCart()
  },

  // 减少商品数量
  reduceGoods(goodsId) {
    const index = this.cartList.findIndex(item => item.goods_id === goodsId)
    if (index !== -1) {
      if (this.cartList[index].quantity > 1) {
        this.cartList[index].quantity--
      } else {
        this.cartList.splice(index, 1)
      }
    }
    this.saveCart()
  },

  // 删除商品
  deleteGoods(goodsId) {
    const index = this.cartList.findIndex(item => item.goods_id === goodsId)
    if (index !== -1) {
      this.cartList.splice(index, 1)
    }
    this.saveCart()
  },

  // 清空购物车
  clearCart() {
    this.cartList = []
    this.saveCart()
  },

  // 保存购物车到本地存储
  saveCart() {
    uni.setStorageSync('cart', JSON.stringify(this.cartList))
  },

  // 从本地存储加载购物车
  loadCart() {
    const cart = uni.getStorageSync('cart')
    if (cart) {
      this.cartList = JSON.parse(cart)
    }
  }
})
```

1.2 商品API调用代码

文件位置: `api/goods.js`

```
import request from '@utils/request'

// 获取商品列表
export const getGoodsList = (params) => {
  return request({
    url: '/goods/list',
    method: 'GET',
    params
  })
}

// 获取商品分类
export const getGoodsCategories = () => {
  return request({
    url: '/categories',
    method: 'GET'
  })
}

// 获取商品详情
export const getGoodsDetail = (goodsId) => {
  return request({
    url: `/goods/${goodsId}`,
    method: 'GET'
  })
}
```

1.3 订单API调用代码

文件位置: `api/order.js`

```
import request from '@utils/request'

// 创建订单
export const createOrder = (data) => {
  return request({
    url: '/orders/create',
    method: 'POST',
    data
  })
}

// 获取订单列表
export const getOrderList = (params) => {
  return request({
```

```
    url: '/orders',
    method: 'GET',
    params
  })
}

// 取消订单
export const cancelOrder = (orderId) => {
  return request({
    url: `/orders/${orderId}/cancel`,
    method: 'POST'
  })
}

// 确认收货
export const confirmReceipt = (orderId) => {
  return request({
    url: `/orders/${orderId}/confirm`,
    method: 'POST'
  })
}
```

1.4 请求封装代码

文件位置: `utils/request.js`

```
import { getToken } from '@utils/token'

const BASE_URL = 'http://localhost:3000/api'

const request = (options) => {
  return new Promise((resolve, reject) => {
    // 显示Loading
    uni.showLoading({
      title: '加载中...',
      mask: true
    })

    // 获取Token
    const token = getToken()

    uni.request({
      url: BASE_URL + options.url,
      method: options.method || 'GET',
      data: options.data || {},
      params: options.params || {},
      header: {
        'Content-Type': 'application/json',
        'Authorization': token ? `Bearer ${token}` : ''
      },
    },
```

```
    success: (res) => {
      uni.hideLoading()

      if (res.statusCode === 200) {
        if (res.data.code === 200) {
          resolve(res.data.data)
        } else {
          uni.showToast({
            title: res.data.message || '请求失败',
            icon: 'none'
          })
          reject(res.data)
        }
      } else {
        uni.showToast({
          title: '网络错误',
          icon: 'none'
        })
        reject(res)
      }
    },
    fail: (err) => {
      uni.hideLoading()
      uni.showToast({
        title: '网络请求失败',
        icon: 'none'
      })
      reject(err)
    }
  })
})
})
}

export default request
```

1.5 首页商品列表代码

文件位置: `pages/index/index.vue`

```
<template>
  <view class="container">
    <!-- 商家信息 -->
    <view class="merchant-info">
      <image :src="merchant.avatar" class="merchant-avatar" />
      <view class="merchant-detail">
        <text class="merchant-name">{{ merchant.name }}</text>
        <view class="merchant-stats">
          <text class="rating">★{{ merchant.rating }}分</text>
          <text class="sales">月售{{ merchant.monthlySales }}</text>
        </view>
      </view>
    </view>
  </view>
</template>
```

```

    </view>
  </view>

  <!-- 地址选择 -->
  <view class="address-bar" @click="goToAddress">
    <text class="address-text">{{ currentAddress }}</text>
    <text class="arrow">></text>
  </view>

  <!-- 商品分类 -->
  <scroll-view class="category-scroll" scroll-x>
    <view
      v-for="category in categories"
      :key="category.id"
      class="category-item"
      :class="{ active: selectedCategory === category.id }"
      @click="selectCategory(category.id)"
    >
      {{ category.name }}
    </view>
  </scroll-view>

  <!-- 商品列表 -->
  <view class="goods-list">
    <view v-for="goods in filteredGoods" :key="goods.id" class="goods-item">
      <image :src="goods.image || defaultImage" class="goods-image" />
      <view class="goods-info">
        <text class="goods-name">{{ goods.name }}</text>
        <text class="goods-desc">{{ goods.description }}</text>
        <view class="goods-bottom">
          <text class="goods-price">¥{{ goods.price }}</text>
          <view class="add-btn" @click="addToCart(goods)">+</view>
        </view>
      </view>
    </view>
  </view>

  <!-- 底部购物车 -->
  <view class="cart-bar">
    <view class="cart-info" @click="toggleCart">
      <text class="cart-count">{{ cartStore.totalCount }}</text>
      <text class="cart-total">¥{{ cartStore.totalAmount.toFixed(2) }}</text>
    </view>
    <button class="checkout-btn" @click="goToCheckout">去结算</button>
  </view>
</template>

<script setup>
import { ref, computed, onMounted } from 'vue'
import { useCartStore } from '@/store/cart'
import { getGoodsList, getGoodsCategories } from '@/api/goods'

const cartStore = useCartStore()

```

```
// 商家信息
const merchant = ref({
  name: '城院\''城小二\''求真1+2幢',
  rating: 5.0,
  monthlySales: 999,
  avatar: '/static/images/logo.png'
})

// 当前地址
const currentAddress = ref('请选择收货地址')

// 分类列表
const categories = ref([])
const selectedCategory = ref(0)

// 商品列表
const goodsList = ref([])
const defaultImage = '/static/images/default-goods.png'

// 过滤后的商品
const filteredGoods = computed(() => {
  if (selectedCategory.value === 0) {
    return goodsList.value
  }
  return goodsList.value.filter(goods => goods.category_id ===
selectedCategory.value)
})

// 加载分类
const loadCategories = async () => {
  try {
    const data = await getGoodsCategories()
    categories.value = [{ id: 0, name: '全部' }, ...data]
  } catch (error) {
    console.error('加载分类失败:', error)
  }
}

// 加载商品
const loadGoods = async () => {
  try {
    const data = await getGoodsList()
    goodsList.value = data
  } catch (error) {
    console.error('加载商品失败:', error)
  }
}

// 选择分类
const selectCategory = (categoryId) => {
  selectedCategory.value = categoryId
}
```

```
// 添加到购物车
const addToCart = (goods) => {
  cartStore.addGoods(goods)
  uni.showToast({
    title: '已添加到购物车',
    icon: 'success'
  })
}

// 跳转地址页
const goToAddress = () => {
  uni.navigateTo({ url: '/pages/address/address?mode=select' })
}

// 跳转结算页
const goToCheckout = () => {
  if (cartStore.isEmpty) {
    uni.showToast({
      title: '购物车为空',
      icon: 'none'
    })
    return
  }
  uni.navigateTo({ url: '/pages/checkout/checkout' })
}

onMounted(() => {
  cartStore.loadCart()
  loadCategories()
  loadGoods()
})
</script>

<style scoped>
.container {
  padding-bottom: 120rpx;
}

.merchant-info {
  display: flex;
  align-items: center;
  padding: 30rpx;
  background: #fff;
}

.merchant-avatar {
  width: 80rpx;
  height: 80rpx;
  border-radius: 50%;
}

.merchant-detail {
  margin-left: 20rpx;
}
```



```
.merchant-name {
  font-size: 32rpx;
  font-weight: bold;
}

.merchant-stats {
  margin-top: 10rpx;
}

.rating, .sales {
  font-size: 24rpx;
  color: #666;
  margin-right: 20rpx;
}

.address-bar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20rpx 30rpx;
  background: #fff;
  border-top: 1rpx solid #eee;
}

.category-scroll {
  white-space: nowrap;
  padding: 20rpx 30rpx;
  background: #fff;
}

.category-item {
  display: inline-block;
  padding: 10rpx 30rpx;
  margin-right: 20rpx;
  border-radius: 30rpx;
  background: #f5f5f5;
  font-size: 28rpx;
}

.category-item.active {
  background: #ff6b6b;
  color: #fff;
}

.goods-list {
  padding: 20rpx 30rpx;
}

.goods-item {
  display: flex;
  padding: 30rpx;
  margin-bottom: 20rpx;
  background: #fff;
}
```

```
border-radius: 16rpx;
}

.goods-image {
width: 160rpx;
height: 160rpx;
border-radius: 8rpx;
}

.goods-info {
flex: 1;
margin-left: 20rpx;
display: flex;
flex-direction: column;
justify-content: space-between;
}

.goods-name {
font-size: 30rpx;
font-weight: bold;
}

.goods-desc {
font-size: 24rpx;
color: #999;
}

.goods-bottom {
display: flex;
justify-content: space-between;
align-items: center;
}

.goods-price {
font-size: 32rpx;
color: #ff6b6b;
font-weight: bold;
}

.add-btn {
width: 50rpx;
height: 50rpx;
line-height: 50rpx;
text-align: center;
background: #ff6b6b;
color: #fff;
border-radius: 50%;
font-size: 36rpx;
}

.cart-bar {
position: fixed;
bottom: 0;
left: 0;
```

```
    right: 0;
    display: flex;
    align-items: center;
    padding: 20rpx 30rpx;
    background: #fff;
    box-shadow: 0 -2rpx 10rpx rgba(0,0,0,0.1);
  }

  .cart-info {
    flex: 1;
  }

  .cart-count {
    font-size: 32rpx;
    font-weight: bold;
  }

  .cart-total {
    margin-left: 20rpx;
    font-size: 28rpx;
    color: #666;
  }

  .checkout-btn {
    padding: 15rpx 50rpx;
    background: #ff6b6b;
    color: #fff;
    border-radius: 30rpx;
  }
</style>
```

二、后端API接口清单

2.1 商品管理接口

```
// 获取商品列表
GET /api/goods/list
Query参数: {
  page: 1,
  pageSize: 10,
  category_id: 1,
  status: 1,
  keyword: '商品名称'
}

// 创建商品
POST /api/goods/create
Body: {
  name: '商品名称',
  category_id: 1,
```

```
    price: 19.99,  
    stock: 100,  
    description: '商品描述',  
    status: 1  
}  
  
// 更新商品  
PUT /api/goods/update/:id  
Body: {  
    name: '商品名称',  
    category_id: 1,  
    price: 19.99,  
    stock: 100,  
    description: '商品描述',  
    status: 1  
}  
  
// 删除商品  
DELETE /api/goods/delete/:id  
  
// 批量删除商品  
POST /api/goods/batch  
Body: {  
    ids: [1, 2, 3]  
}  
  
// 更新商品状态  
PUT /api/goods/update-status/:id  
Body: {  
    status: 1  
}
```

2.2 订单管理接口

```
// 创建订单  
POST /api/orders/create  
Body: {  
    address_id: 1,  
    goods_list: [  
        {  
            goods_id: 1,  
            quantity: 2,  
            price: 19.99  
        }  
    ],  
    total_amount: 39.98  
}  
  
// 获取订单列表  
GET /api/orders
```

```
Query参数: {
  page: 1,
  pageSize: 10,
  status: 1,
  user_phone: '13800138000',
  start_time: '2025-01-01',
  end_time: '2025-01-31'
}

// 获取订单详情
GET /api/orders/:id

// 取消订单
POST /api/orders/:id/cancel

// 确认收货
POST /api/orders/:id/confirm

// 更新订单状态
PATCH /api/admin/orders/status/:orderId
Body: {
  status: 2,
  delivery_person_id: 1
}

// 导出订单
GET /api/admin/orders/export
Query参数: {
  status: 1,
  start_time: '2025-01-01',
  end_time: '2025-01-31'
}
```

2.3 分类管理接口

```
// 获取分类列表
GET /api/categories

// 创建分类
POST /api/admin/categories/create
Body: {
  name: '分类名称',
  description: '分类描述',
  sort: 0
}

// 更新分类
PUT /api/admin/categories/update/:id
Body: {
  name: '分类名称',
```

```
    description: '分类描述',
    sort: 0
  }

// 删除分类
DELETE /api/admin/categories/delete/:id
```

2.4 用户管理接口

```
// 用户登录
POST /api/user/login
Body: {
  phone: '13800138000',
  password: '123456'
}

// 获取用户信息
GET /api/user/info

// 获取用户订单
GET /api/admin/users/:userId/orders
Query参数: {
  page: 1,
  pageSize: 10
}

// 更新用户状态
PATCH /api/admin/users/status/:id
Body: {
  status: 1
}
```

三、管理端代码清单 (Vue3)

3.1 商品管理页面

文件位置: `src/views/Goods/Goods.vue`

```
<template>
  <div class="goods-container">
    <!-- 操作按钮 -->
    <el-button type="primary" @click="handleAdd">新增商品</el-button>
    <el-button type="danger" :disabled="selectedIds.length === 0"
      @click="handleBatchDelete">
      批量删除
    </el-button>
  </div>
</template>
```

```
<!-- 搜索表单 -->
<el-form :inline="true" :model="searchForm" class="search-form">
  <el-form-item label="商品名称">
    <el-input v-model="searchForm.keyword" placeholder="请输入商品名称" />
  </el-form-item>
  <el-form-item label="分类">
    <el-select v-model="searchForm.categoryId" placeholder="请选择分类">
      <el-option label="全部" :value="0" />
      <el-option
        v-for="cat in categories"
        :key="cat.id"
        :label="cat.name"
        :value="cat.id"
      />
    </el-select>
  </el-form-item>
  <el-form-item label="状态">
    <el-select v-model="searchForm.status" placeholder="请选择状态">
      <el-option label="全部" :value="" />
      <el-option label="上架" :value="1" />
      <el-option label="下架" :value="0" />
    </el-select>
  </el-form-item>
  <el-form-item>
    <el-button type="primary" @click="handleSearch">搜索</el-button>
    <el-button @click="handleReset">重置</el-button>
  </el-form-item>
</el-form>

<!-- 商品列表表格 -->
<el-table
  :data="goodsList"
  @selection-change="handleSelectionChange"
  style="width: 100%"
>
  <el-table-column type="selection" width="55" />
  <el-table-column prop="id" label="商品ID" width="80" />
  <el-table-column prop="name" label="商品名称" />
  <el-table-column prop="category_name" label="分类" width="120" />
  <el-table-column prop="price" label="价格" width="100">
    <template #default="{ row }">
      ¥{{ row.price.toFixed(2) }}
    </template>
  </el-table-column>
  <el-table-column prop="stock" label="库存" width="100" />
  <el-table-column prop="status" label="状态" width="100">
    <template #default="{ row }">
      <el-switch
        v-model="row.status"
        :active-value="1"
        :inactive-value="0"
        @change="handleStatusChange(row)"
      />
    </template>
  </el-table-column>
</table>
```

```
</el-table-column>
<el-table-column prop="created_at" label="创建时间" width="180" />
<el-table-column label="操作" width="150" fixed="right">
  <template #default="{ row }">
    <el-button type="primary" size="small" @click="handleEdit(row)">
      编辑
    </el-button>
    <el-button type="danger" size="small" @click="handleDelete(row.id)">
      删除
    </el-button>
  </template>
</el-table-column>
</el-table>

<!-- 分页 -->
<el-pagination
  v-model:current-page="pagination.page"
  v-model:page-size="pagination.pageSize"
  :total="pagination.total"
  :page-sizes="[10, 20, 50, 100]"
  layout="total, sizes, prev, pager, next, jumper"
  @size-change="loadGoodsList"
  @current-change="loadGoodsList"
/>

<!-- 商品表单对话框 -->
<el-dialog
  v-model="dialogVisible"
  :title="isEdit ? '编辑商品' : '新增商品'"
  width="500px"
>
  <el-form :model="form" :rules="rules" ref="formRef" label-width="80px">
    <el-form-item label="商品名称" prop="name">
      <el-input v-model="form.name" placeholder="请输入商品名称" />
    </el-form-item>
    <el-form-item label="分类" prop="categoryId">
      <el-select v-model="form.categoryId" placeholder="请选择分类">
        <el-option
          v-for="cat in categories"
          :key="cat.id"
          :label="cat.name"
          :value="cat.id"
        />
      </el-select>
    </el-form-item>
    <el-form-item label="价格" prop="price">
      <el-input-number
        v-model="form.price"
        :min="0"
        :step="0.01"
        :precision="2"
        controls-position="right"
      />
    </el-form-item>
  </el-form>
</el-dialog>
```



```
<el-form-item label="库存" prop="stock">
  <el-input-number v-model="form.stock" :min="0" controls-position="right"
/>
</el-form-item>
<el-form-item label="描述" prop="description">
  <el-input
    v-model="form.description"
    type="textarea"
    :rows="3"
    placeholder="请输入商品描述"
  />
</el-form-item>
<el-form-item label="状态" prop="status">
  <el-radio-group v-model="form.status">
    <el-radio :label="1">上架</el-radio>
    <el-radio :label="0">下架</el-radio>
  </el-radio-group>
</el-form-item>
</el-form>
<template #footer>
  <el-button @click="dialogVisible = false">取消</el-button>
  <el-button type="primary" @click="handleSubmit">确定</el-button>
</template>
</el-dialog>
</div>
</template>

<script setup>
import { ref, reactive, onMounted } from 'vue'
import { ElMessage, ElMessageBox } from 'element-plus'
import request from '@/utils/request'

// 搜索表单
const searchForm = reactive({
  keyword: '',
  categoryId: 0,
  status: ''
})

// 商品列表
const goodsList = ref([])
const selectedIds = ref([])

// 分类列表
const categories = ref([])

// 分页
const pagination = reactive({
  page: 1,
  pageSize: 10,
  total: 0
})

// 对话框
```

```
const dialogVisible = ref(false)
const isEdit = ref(false)

// 表单
const formRef = ref(null)
const form = reactive({
  name: '',
  categoryId: null,
  price: 0,
  stock: 0,
  description: '',
  status: 1
})

// 表单验证规则
const rules = {
  name: [{ required: true, message: '请输入商品名称', trigger: 'blur' }],
  categoryId: [{ required: true, message: '请选择分类', trigger: 'change' }],
  price: [{ required: true, message: '请输入价格', trigger: 'blur' }],
  stock: [{ required: true, message: '请输入库存', trigger: 'blur' }]
}

// 加载分类列表
const loadCategories = async () => {
  try {
    const res = await request.get('/admin/categories/list')
    categories.value = res.data
  } catch (error) {
    console.error('加载分类失败:', error)
  }
}

// 加载商品列表
const loadGoodsList = async () => {
  try {
    const res = await request.get('/admin/goods/list', {
      params: {
        page: pagination.page,
        pageSize: pagination.pageSize,
        ...searchForm
      }
    })
    goodsList.value = res.data.list
    pagination.total = res.data.total
  } catch (error) {
    console.error('加载商品失败:', error)
  }
}

// 搜索
const handleSearch = () => {
  pagination.page = 1
  loadGoodsList()
}
```

```
// 重置
const handleReset = () => {
  Object.assign(searchForm, {
    keyword: '',
    categoryId: 0,
    status: ''
  })
  handleSearch()
}

// 新增商品
const handleAdd = () => {
  isEdit.value = false
  Object.assign(form, {
    name: '',
    categoryId: null,
    price: 0,
    stock: 0,
    description: '',
    status: 1
  })
  dialogVisible.value = true
}

// 编辑商品
const handleEdit = (row) => {
  isEdit.value = true
  Object.assign(form, {
    id: row.id,
    name: row.name,
    categoryId: row.category_id,
    price: row.price,
    stock: row.stock,
    description: row.description,
    status: row.status
  })
  dialogVisible.value = true
}

// 提交表单
const handleSubmit = async () => {
  try {
    await formRef.value.validate()
    if (isEdit.value) {
      await request.put(`/admin/goods/update/${form.id}`, form)
      ElMessage.success('更新成功')
    } else {
      await request.post('/admin/goods/create', form)
      ElMessage.success('添加成功')
    }
    dialogVisible.value = false
    loadGoodsList()
  } catch (error) {
```

```
    if (error !== false) {
      ElMessage.error('操作失败')
    }
  }
}

// 删除商品
const handleDelete = async (id) => {
  try {
    await ElMessageBox.confirm('确定要删除该商品吗?', '提示', {
      type: 'warning'
    })
    await request.delete(`/admin/goods/delete/${id}`)
    ElMessage.success('删除成功')
    loadGoodsList()
  } catch (error) {
    if (error !== 'cancel') {
      ElMessage.error('删除失败')
    }
  }
}

// 批量删除
const handleBatchDelete = async () => {
  try {
    await ElMessageBox.confirm(`确定要删除选中的${selectedIds.value.length}个商品吗?`, '提示', {
      type: 'warning'
    })
    await request.post('/admin/goods/batch', { ids: selectedIds.value })
    ElMessage.success('批量删除成功')
    loadGoodsList()
  } catch (error) {
    if (error !== 'cancel') {
      ElMessage.error('批量删除失败')
    }
  }
}

// 状态改变
const handleStatusChange = async (row) => {
  try {
    await request.put(`/admin/goods/update-status/${row.id}`, {
      status: row.status
    })
    ElMessage.success('状态更新成功')
  } catch (error) {
    row.status = row.status === 1 ? 0 : 1
    ElMessage.error('状态更新失败')
  }
}

// 选择改变
const handleSelectionChange = (selection) => {
```

```
    selectedIds.value = selection.map(item => item.id)
  }

  onMounted(() => {
    loadCategories()
    loadGoodsList()
  })
</script>

<style scoped>
.goods-container {
  padding: 20px;
}

.search-form {
  margin: 20px 0;
}
</style>
```

3.2 请求封装代码

文件位置: `src/utils/request.js`

```
import axios from 'axios'
import { ElMessage } from 'element-plus'

const request = axios.create({
  baseURL: 'http://localhost:3000/api',
  timeout: 10000
})

// 请求拦截器
request.interceptors.request.use(
  config => {
    const token = localStorage.getItem('admin_token')
    if (token) {
      config.headers.Authorization = `Bearer ${token}`
    }
    return config
  },
  error => {
    return Promise.reject(error)
  }
)

// 响应拦截器
request.interceptors.response.use(
  response => {
    const res = response.data
    if (res.code === 200) {

```

```
        return res
      } else {
        ElMessage.error(res.message || '请求失败')
        return Promise.reject(res)
      }
    },
    error => {
      ElMessage.error(error.message || '网络错误')
      return Promise.reject(error)
    }
  )

export default request
```

版权所有 © 2025 城小二校园外卖系统